Persistent Scene Change Detection & Localisation

A Computer Vision Project

Jiyaro Joseph Joseph S. Mathew

Government Model Engineering College, Kochi

July 2026

Jiyaro Joseph (EC7B)
jiyarojoseph27.mec@gmail.com

Joseph S. Mathew (CS7A)
josephsmathew.mec@gmail.com

Problem Statement

- **Task:** Detect objects *left behind* in fixed-camera surveillance video.
- **Exclude:** transient objects (pedestrians, shadows, moving vehicles).
- **p/c Rule:**
  - $p = 60$ s – object must be present for 60 s before flagging starts.
  - $c = 90$ s – flag for $(c - p) = 30$ s, then stop (object assimilated).
- **Output:** Binary masks per second; white = persistent change.
- **Evaluation:** Pixel-level F1 score (balances precision & recall).

Example: Original Frame and Output Mask

Original frame

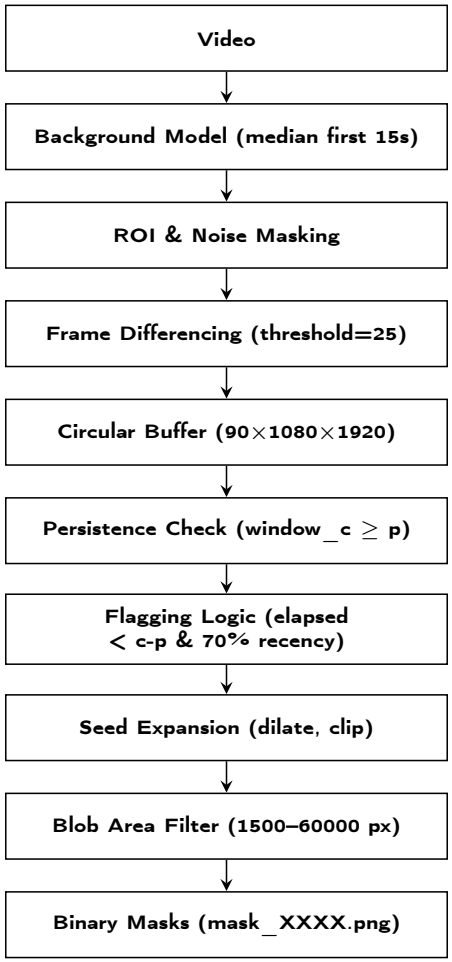
Final binary mask (persistent change)

White pixels indicate the detected persistent object; black is background / ignored.

Approach Selection

Approach	Why rejected?	Chosen?
MOG2/KNN	Adapts to scene – absorbs persistent objects	×
U-Net / Deep Learning	Only 5 videos – massive overfitting	×
SAM2	No temporal logic (p/c)	×
Object tracking	RAM crash, occlusions corrupt IDs	×
Classical CV	Full control, memory-efficient, no training	✓

Pipeline Architecture



Background Modeling

- **Method:** median of first 15 s frames.

```
background = np.median(frames[:int(fps*15)], axis=0).astype(np.uint8)
```

- **Why median?** Robust to outliers (pedestrians) – mean would create ghosts.
- **Adaptive duration:** $\min(15, \max(8, 10\% \text{ of duration}))$.

ROI & Noise Masking

- **Top 20% excluded:** sky, trees – only swaying noise, no objects.
- **Automatic noise masking:**
  - Measure noise in first clean seconds.
  - Pixels noisy $>50\%$ of time $\rightarrow$ structural noise.
  - Mask only large blobs ($\geq$5000 px) – preserves possible object locations.

```
region = cv2.bitwise_and(region, cv2.bitwise_not(filtered))
```

Frame Differencing

1. Sample one frame per second (midpoint) – avoids motion blur.
2. Absolute difference against background, apply ROI.
3. Threshold = 25.
4. Morphological cleanup: open (removes small noise) then close (fills holes).

```
diff = cv2.absdiff(gray, background)
_, cur = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
cur = cv2.morphologyEx(cur, cv2.MORPH_OPEN, kernel)
cur = cv2.morphologyEx(cur, cv2.MORPH_CLOSE, kernel)
```

Circular Buffer (Core Innovation)

- **Memory problem:**
  - Full video history (e.g., 417 s) = 3.4 GB (naive)
  - 90-second sliding window (int32) = 746 MB
- **Our solution:** circular buffer using `uint8` $\rightarrow$ constant **186 MB**, independent of video length.
- Running sum `window_c` = exactly last 90 frames.
- O(1) update per second – no loop over past frames.

```
buf = np.zeros((c, h, w), dtype=np.uint8)
slot = sec % c
window_c -= buf[slot]     # remove oldest
buf[slot] = cur_bin       # overwrite
window_c += cur_bin       # add newest
```

Persistence Logic & Flagging

- `first_active` map: records the second when pixel's count $\geq p$ (60).
- Flagging window: exactly $(c - p) = 30$ seconds after activation.
- **70% recency filter:** ensures pixel is present in 70% of the last 60 s – separates stationary objects from repeated pedestrians.

```
elapsed = sec - first_active
seeds = (first_active>=0) & (elapsed>=0) & (elapsed<30)
recent = sum(buf[(sec-i)%c] for i in range(p))
seeds &= (recent >= int(p*0.70))
```

Seed Expansion & Blob Filtering

- Morphological close, open to clean seeds.
- Dilate (15×15) to expand to full object.
- Clip expansion to the actual difference region (`diff_bin`).
- **Blob area filter (data-driven):**
  - False positives: ≤ 1521 px; true objects: ≥ 4091 px.
  - Set `min_area` $= 1500$, `max_area` $= 60000$.

```
expanded = cv2.dilate(seeds, kernel)
result = cv2.bitwise_and(expanded, diff_bin)
```

Results

Video	Scene	Precision	Recall	F1
video_1	trolley bags, busy	0.163	0.735	0.267
video_2	small bag (tiny)	0.000	0.000	0.000
video_3	motorcycle, clean	0.828	0.692	**0.754**
video_4	thela cart, pedestrians	0.130	0.655	0.217
video_5	dense parking	0.104	0.318	0.157
Overall		**0.161**	**0.507**	**0.244**

- Best on video_3: large object, clean background.
- video_2 F1=0 because the bag is only ∼892 px, below `min_area` – accepted trade-off.

Failure Analysis & Lessons

- **Pedestrian accumulation:** Repeated walkers trigger false positives.
  Lesson: Pixel persistence can't separate frequency from duration.
- **Small objects (video_2):** Bag below min_area.
  Lesson: Fixed size thresholds don't generalise; per-video adaptation needed.
- **Stationary pedestrian during background setup:** False detection after they leave.
  Lesson: Background initialisation must handle temporary occlusions.
- **Object tracking backfired:** Centroid tracker crashed; F1 dropped to 0.041.
  Lesson: Simple methods beat complex, brittle approaches.

Future Work

- **SAM2 refinement:** use coarse detection $\rightarrow$ SAM2 for precise boundaries.
- **RAM-efficient object tracking:** streaming tracker with occlusion-aware IDs to reduce false positives.
- **Per-video adaptive thresholding:** set min_area from first 5 clean seconds – would fix video_2.
- **Background update after assimilation:** slowly absorb object after c seconds – handles repeated placements.

Conclusion

- We replaced adaptive background subtraction with a fixed median reference to ensure stationary objects never "disappear" into the background.
- The Circular Buffer architecture converted a 3.4GB history-tracking problem into a 186MB constant-memory process suitable for any video length.
- **Simple + principled** outperforms complex + brittle – circular buffer gave stable 0.244 F1, while object tracking crashed and dropped to 0.041.

Overall F1: 0.244 | **Best video F1: 0.754**